



SCHEDULER · BIN-PACKING · MILP OPTIMIZATION

Оптимизация загрузки серверов в облаке

K8s-like scoring · bin-packing · MILP capacity planning

Демо-проект по методам оптимизации



CONTEXT

Постановка задачи

Что и зачем

- Распределение нагрузок при нестационарном спросе.
- Меньше активных хостов и плотнее упаковка pod'ов.
- Считаем: счётчик активных нод, миграции, score нод.

MAX HOSTS

80

3 пула в config

PLANNERS

02

bin-packing · MILP

SCORING

2

LeastAlloc · ImageLoc

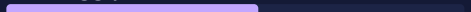


DOMAIN

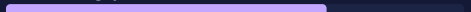
Kubernetes-like кластер

NODE-01

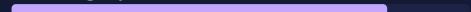
compute-large

CPU 62 %
RAM 55 %
PODS ×3
**NODE-02**

compute-medium

CPU 45 %
RAM 70 %
PODS ×2
**NODE-03**

memory-heavy

CPU 78 %
RAM 82 %
PODS ×3
**NODE****Worker-машина**

Сервер с лимитом CPU/RAM,
на нём запускаются pod'ы.

POD**Минимальная единица**

Контейнер с `cpu_request`
и `memory_request`.

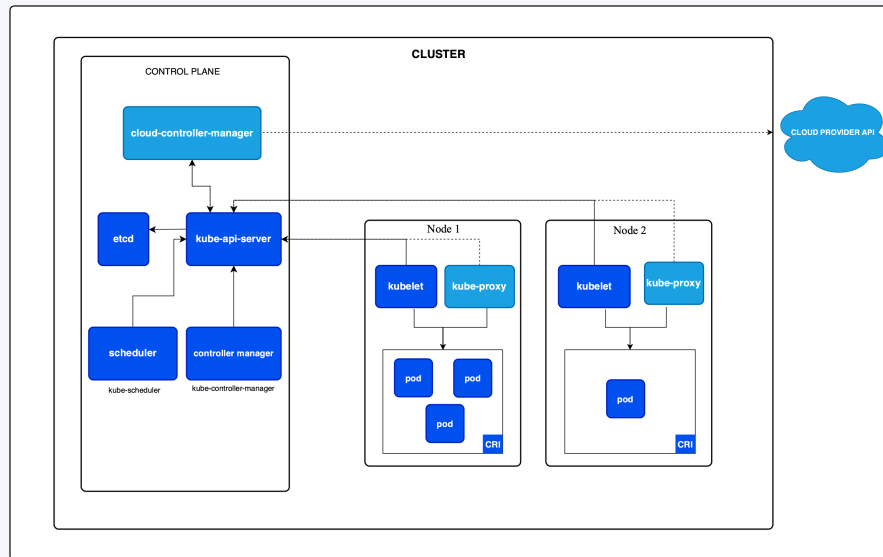
SCHEDULER**Распределитель**

Фильтрует ноды и выбирает
лучшую для каждого pod'а.



ARCHITECTURE

Архитектура Kubernetes





COMPLETE MODEL

Полная постановка задачи

DOMAIN

$$P, H, c_j, m_j, \text{cost}_j, r_i^{\text{cpu}}, r_i^{\text{mem}}$$

P/H — pod'ы и хосты; ёмкость, стоимость, запросы.

VARIABLES

$$x_{ij}, y_j \in \{0, 1\}$$

x: pod i на ноде j. y: нода j активна.

OBJECTIVE

$$\min \sum_{j \in H} \text{cost}_j \cdot y_j$$

Минимум стоимости активного кластера (\$/ч).

ASSIGN

$$\sum_{j \in H} x_{ij} = 1, \quad \forall i \in P$$

Каждый pod — ровно на одной ноде.

CPU

$$\sum_{i \in P} r_i^{\text{cpu}} x_{ij} \leq c_j y_j$$

Σ CPU-запросов $\leq$ ёмкости активной ноды.

RAM

$$\sum_{i \in P} r_i^{\text{mem}} x_{ij} \leq m_j y_j$$

То же для памяти.



BRANCH AND BOUND

Как решается MILP

РЕЛАКСАЦИЯ

$$x_{ij}, y_{p,k} \in [0, 1]$$

Заменяем 0/1 -> отрезок [0,1]. Решается за полиномиальное время.

ГРАНИЦА

$$z_{LP} \leq z_{MILP}^*$$

Оптимум релаксации — нижняя граница для исходной задачи.

СВЯЗЬ

$$\sum_i r_i x_{i,p,k} \leq c_p y_{p,k}$$

При $y=0$ правая часть = 0, значит и $x = 0$.

ВЕТВЛЕНИЕ

$$y_{p,k}^* \notin \{0, 1\} \Rightarrow \{y_{p,k} = 0\} \cup \{y_{p,k} = 1\}$$

Дробный ответ -> две подзадачи: $y=0$ и $y=1$.

ОТСЕЧЕНИЕ

$$\text{add: } a^\top x \leq b \text{ s.t. } z_{LP}^{\text{new}} > z_{LP}$$

Добавляем неравенства, поднимаем нижнюю границу.

ОТСЕВ

$$z_{LP}(\text{node}) \geq z_{\text{best}} \Rightarrow \text{prune}$$

Граница ветви $\geq$ лучшего решения — пропускаем ветку.

СИММЕТРИЯ

$$y_{p,k+1} \leq y_{p,k}$$

Ноды одного пула идентичны -> открываем по порядку, не перебираем перестановки.



GREEDY HEURISTIC

Что используется в проде? Bin-packing!

АЛГОРИТМ

1. Сортируем поды по убыванию (ядра, память) — большие первыми.
2. Для каждого пода — лучшая открытая нода по K8s-like score.
3. Не помещается? — открываем самый дешёвый подходящий шаблон ноды.

✓ Плюсы

- Близок к оптимуму на типичных нагрузках
- Онлайн: можно запускать инкрементально
- Простая реализация, легко отлаживать
- Совместим с K8s scoring: LeastAlloc, ImageLocality
- Cluster Autoscaler — это про bin-packing

✗ Минусы

- Жадный — не видит задачу глобально
- Результат зависит от порядка прихода подов
- Не учитывает взаимодействие подов
- Нет гарантии оптимальности



WORKED EXAMPLE

Пример: bin-packing vs MILP

пулы

L: 50 ядер · 200 ГиБ · \$2/ч · до 2

H: 40 ядер · 400 ГиБ · \$3/ч · до 1

поды

A: 30 ядер · 0 ГиБ

B: 4 ядра · 120 ГиБ

C: 4 ядра · 120 ГиБ

Bin-packing (жадный)

1. Сортируем: A(30), B(4), C(4)
2. A -> открываем L1 (\$2)
3. B -> влезает в L1
4. C -> не влезает, открываем L2 (\$2)
-> $2 \times L = \$4/\text{ч}$

MILP (ветви и границы)

1. ЛП-релаксация: $y_H = 0.95$
2. Ветка $y_H = 1$: всё на H ✓ $z = \$3$
3. Ветка $y_H = 0$: \$4 — хуже
-> $1 \times H = \$3/\text{ч}$

MILP: \$3/ч vs bin-packing: \$4/ч.



SUMMARY

Выводы

PLAN

Планировали

- Сравнить эвристику и MILP
- Оценить выигрыш по стоимости

DONE

Сделано

- Реализованы оба подхода
- Показана разница на примере

NEXT

Дальше

- Симуляция во времени
- Сложная постановка с новыми ограничениями



Спасибо за внимание!

Вопросы?